A 28-nm 88.3-TFLOPS/W POSIT-Approximate-Calculation-Based Digital Computing-in-Memory Macro Incorporating Final-Cycle Fusion and Joint Skipping

Hao Wu, Yong Chen, *Senior Member, IEEE*, Ming Li, Bingxin Zhang, Rui Zhang, Yiyang Yuan, Yiming Yang, Jinshan Yue, *Member, IEEE*, Xinghua Wang, Xiaoran Li, and Feng Zhang, *Senior Member, IEEE*

***Abstract*—Computing-in-memory (CiM) architecture is considered a technology that promises to alleviate the memory wall problem in artificial intelligence (AI) computing. Previous work on digital CiM macros has achieved significant energy efficiency advantages in supporting approximate calculations of traditional integer (INT)/floating point (FP) data. POSIT is an emerging FP number system. Its main characteristic is that it can approach a good tradeoff between data range and accuracy by flexibly adjusting the mantissa bit width. However, current digital CiM macros lack the ability to support approximate calculations of POSIT data. Given this, we propose a POSIT-approximate-calculation-based CiM macro. The three main contributions of our work are as follows: 1) we improve the original POSIT approximate calculation principle and design a calculation logic circuit based on unsigned Radix8+ final-cycle fusion (FCF); 2) we propose a macro-level approximate calculation control strategy to help the CiM macro find a compromise between calculation performance and calculation error; and 3) we design an activation mantissa alignment skipping (AMAS) unit, to help the hardware skip sparse activation bits when calculating POSIT multiplication. A prototype of the CiM macro was fabricated in a 28-nm complementary metal oxide semiconductor (CMOS) and scored a peak macro energy efficiency of 88.3 TFLOPS/W @ POSIT (8, 2) for a voltage supply of 0.55–1.1 V and an operating frequency of 70–280 MHz, a value 2.12× higher than other state-of-the-art POSIT CiM macros.**

***Index Terms*—Approximate calculation, artificial intelligence (AI), complementary metal oxide semiconductor (CMOS), computing-in-memory (CiM), energy efficiency, final-cycle fusion (FCF), joint skipping, multiply accumulation (MAC), neural network (NN), POSIT, Radix8.**

I. Introduction

ALGORITHMS in the form of large models are applicable to a variety of tasks and have increasing requirements for the computing performance and energy efficiency of their computing hardware. Computing-in-memory (CiM) is a technology that has the potential to solve the problem of high power consumption caused by data transfer in artificial intelligence (AI) algorithms. It uses computing circuits near the storage array to directly read and calculate stored data, which can significantly reduce power consumption due to weight data movement in AI workloads [1]. The current CiM macro can effectively support the multiply-accumulate calculation (MAC) for traditional floating point (FP) numbers [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13]. One method is to use an analog-to-digital converter (ADC) as the readout and calculation circuit (i.e., analog CiM) [8], [9], [10], [11], while another approach is to use digital logic circuits to achieve high computational energy efficiency in the inner product computing array (i.e., digital CiM) [2], [3], [4], [5], [6], and a further solution is to combine the advantages of both the digital and analog domains (i.e., hybrid CiM) [12], [13].

POSIT is an emerging floating-point number format [14] that is characterized by a more efficient way of allocating precision across different numerical ranges. It can provide higher precision for values close to one and can appropriately discard some precision for extremely large or small numbers. This precision allocation feature is very suitable for AI algorithms [15], [16] and can achieve better algorithm performance with the same data bit width [19]. Previous researchers have designed efficient computing principles and computing hardware for the special format of POSIT [17], [18], [19], but these schemes did not make full use of the variable precision characteristics of POSIT. For this reason, an approximate calculation method

Received 29 November 2024; revised 6 April 2025 and 28 May 2025; accepted 11 July 2025. Date of publication 5 August 2025; date of current version 30 March 2026. This article was approved by Associate Editor Jie Gu. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB4402400 and in part by the National Natural Science Foundation of China under Grant 92464201 and Grant U2341218. *(Corresponding authors: Yong Chen; Feng Zhang.)*

Hao Wu, Ming Li, Bingxin Zhang, Rui Zhang, Yiyang Yuan, Jinshan Yue, and Feng Zhang are with the Key Laboratory of Microelectronics Devices and Integrated Technology, Institute of Microelectronics, Chinese Academy of Sciences, Beijing 100029, China, and also with the University of the Chinese Academy of Sciences, Beijing 100049, China (e-mail: zhangfeng_ime@ime.ac.cn).

Yong Chen is with the Department of Electronic Engineering and the State Key Laboratory of Space Network and Communications, Tsinghua University, Beijing 10084, China (e-mail: ychen_ee@tsinghua.edu.cn).

Yiming Yang, Xinghua Wang, and Xiaoran Li are with the School of Information and Electronics, Beijing Institute of Technology, Beijing 100081, China.

Digital Object Identifier 10.1109/JSSC.2025.3590632.

0018-9200 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:57 UTC from IEEE Xplore. Restrictions apply.

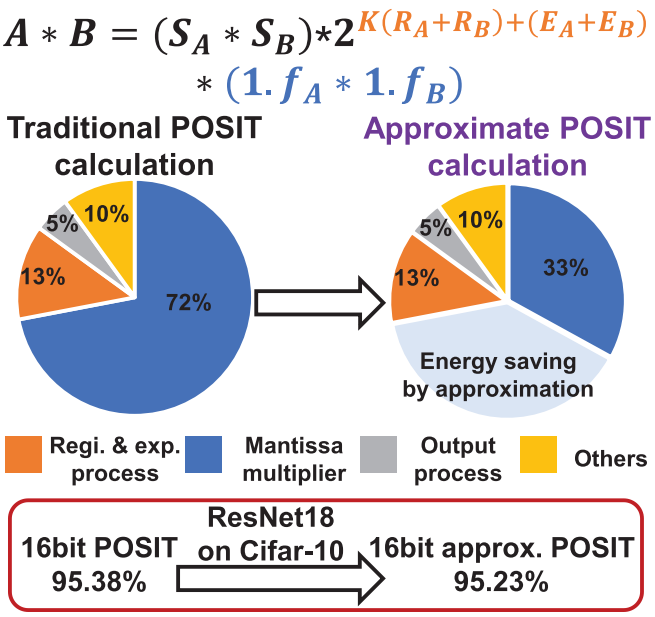


Fig. 1. Approximate calculation in the POSIT number system.

with a small loss of algorithmic accuracy is needed to reduce the power requirements for mantissa multiplication, which accounts for a large proportion of POSIT calculations, as shown in Fig. 1. The following is a detailed analysis of the challenges of achieving high energy efficiency POSIT approximate calculation on the CiM architecture shown in Fig. 2.

Challenge 1: There is a need for a CiM calculation circuit that is suitable for the POSIT mantissa to further reduce the number of calculation cycles and improve energy efficiency.

Challenge 2: The fixed array structure of the CiM macro makes it impossible to flexibly skip different computing cycles, thus making it hard for CiM macros to gain performance benefits from approximate calculations.

Challenge 3: There are many sparse bits in the longer mantissa part of the POSIT activation data. These sparse bits take up additional computing cycles, resulting in a decrease in computing performance.

In this article, we propose a digital CiM macro that integrates the approximate computation of POSIT for two different POSIT precisions: (8, 2) and (16, 2). It is designed to surmount the three challenges described above concerning the approximate POSIT calculation on the CiM macro. To our knowledge, we first time innovate the POSITapproximate calculation function in a digital CIM macro.

To solve Challenge 1, we propose a new digital CiM macro that supports POSIT calculations with different precisions of (8, 2) and (16, 2), which can determine which parts of the mantissa can be discarded before performing POSIT floating-point MAC calculations. It uses unsigned Radix8-based calculation logic circuits with final-cycle fusion (FCF) to further reduce the number of calculation cycles and improve the energy efficiency by 2.19–2.79×.

To address Challenge 2, we devise a macro-level approximate calculation control (MACC) unit to dynamically adjust the number of cycles to achieve a speedup of 1.89× with only a 3.24% mean absolute percentage error (MAPE) loss.

To tackle Challenge 3, we propose an activation mantissa alignment skipping (AMAS) unit that helps to reduce the cycle numbers from sparse input code, obtaining a 1.41× speed improvement.

II. Background and Motivation

A. Advantages of POSIT Floating-Point Numbers

POSIT is a new floating-point number system. As shown in Fig. 3(a), its main characteristic is that the bit width of the exponent part is fixed, as is the sum of the regime width and the mantissa width. However, the bit width of the regime and mantissa parts can be flexibly adjusted to provide different degrees of accuracy within different data size ranges. In the POSIT floating-point format (N, ES), N is the bit width of the entire data and ES is the width of the exponent part. Compared with traditional floating-point numbers, POSIT has an added regime part, which starts with continuous 0 bits or continuous 1 bits and ends where the last bit is inverted. The principle used to calculate the regime value is as follows. Let m be the bit width of the same part of the regime. When the first bit of this part is zero, the value of the regime is $-m$, and when the first bit of this part is one, the value of R is $m-1$. The bit width of the exponent in POSIT is fixed at 2 bits, so the value of ES is two according to the latest POSIT standard, issued in 2022 [29]. Finally, we can summarize the value calculation formula for all POSIT numbers as shown in the following equation:

$$\text{value} = \begin{cases} 0, & 000,\ldots,0 \\ \pm\infty, & 100,\ldots,0 \\ (-1)^{\text{sign}} \times \left(2^{K}\right)^{R} \times 2^{E} \times 1 \cdot M, & K = 2^{\text{ES}}, \\ & \text{otherwise} \end{cases} \tag{1}$$

where E is the value of the exponent and M is the value of the mantissa.

In Fig. 4(a), we first compare the accuracy of the two data formats. It can be seen that although the data bit width of POSIT (8, 2) is half that of BF16, it can achieve different accuracy adjustments in different data intervals. Compared with the very uniform accuracy of BF16, the accuracy distribution of POSIT (8, 2) is more consistent with the distribution of activation and weight data in the algorithm. Fig. 4(b) shows the comparison of the hardware implementation cost between BF16 and POSIT in 28 nm. From the data in Fig. 4(b), we can see that compared with BF16, the accuracy loss of POSIT (8, 2) is only 0.08%, but the hardware cost is only 30%–40% of BF16. POSIT (16, 2) precision has the highest algorithm accuracy. However, due to its larger mantissa width, its computational efficiency is lower than BF16 when the mantissa exceeds 8 bits. To this end, a new optimization technique for mantissa needs to be proposed to reduce the computational cost of the POSIT (16, 2).

After the above comparison, we can find that POSIT offers significant advantages compared with traditional integer (INT) types and floating-point types, especially in AI calculations

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:57 UTC from IEEE Xplore. Restrictions apply.

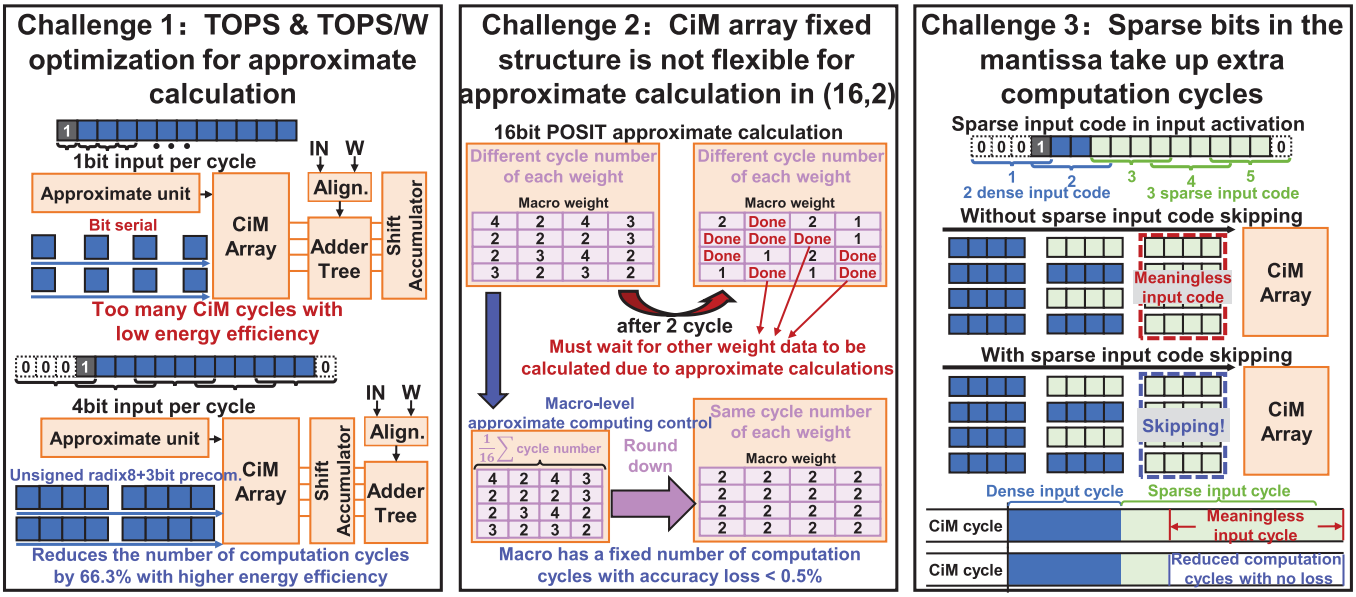


Fig. 2. Key challenges in implementing POSIT floating-point approximate calculations on CiM architectures.

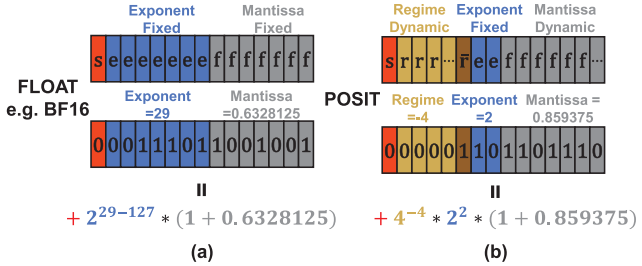


Fig. 3. (a) Traditional and (b) POSIT floating-point numbers with examples.

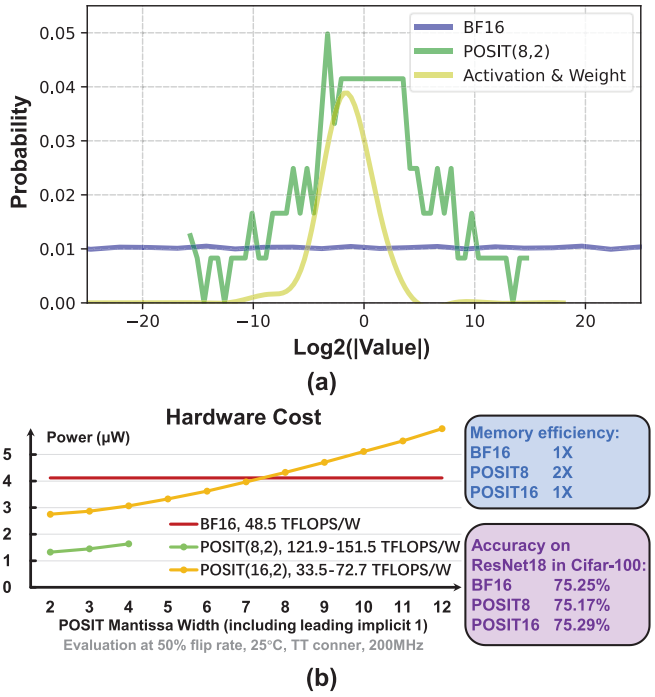


Fig. 4. (a) Comparison of data distribution probability between BF16 and POSIT (8, 2). (b) Comparison of hardware cost, memory efficiency, and accuracy between BF16 and POSIT.

[19]. It achieves a better balance between representation range and accuracy. Most of the data in the AI workload are distributed near a value of zero [27], [28]; by using dynamic bit allocation technology, POSIT can obtain more mantissa bit width in the vicinity of zero, meaning that it is very suitable for AI computing tasks [15], [16]. Previous researchers have proposed many techniques to enable hardware to support the flexible variable-length feature of POSIT and have achieved good results [17], [18], [19]. However, these schemes did not include special hardware optimization for the dynamic length characteristics of POSIT.

B. Design of an Approximate-Calculation CiM Macro

Approximate computing is a technology that can help AI chips achieve further breakthroughs in energy efficiency. Its main principle is to reduce the cost of computing hardware while sacrificing computing accuracy. Recent work has demonstrated several different techniques for implementing approximate computing in CiM architectures in Fig. 5. He et al. [7] proposed a CiM macro that did not require multiplication calculations. The $L1$ distance was used as an approximate substitute for the $L2$ distance to reduce the large proportion of the MAC computing cost in AI loads. In [25], a general-purpose approximate compressor was proposed that used OR/AND gates to replace the full adders (FAs) in traditional multipliers, ultimately achieving high area efficiency and low power consumption for approximate calculations. In [10], hybrid exact-approximate multipliers were presented, where exact multipliers were used for the high-bit part and approximate multipliers for the low-bit part, to achieve a good tradeoff between calculation accuracy and calculation cost.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:57 UTC from IEEE Xplore. Restrictions apply.

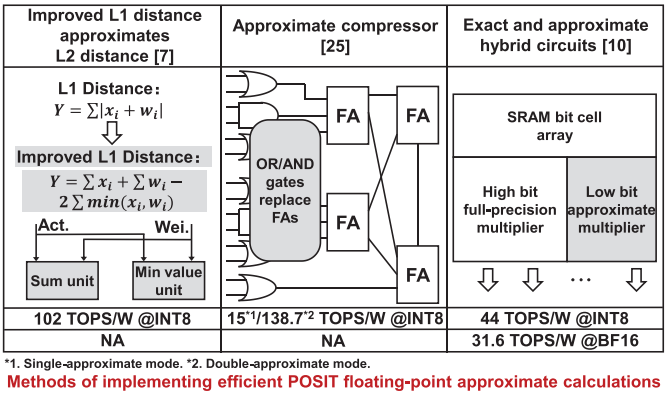


Fig. 5. Comparison of previously proposed CiM macro solutions supporting approximate computing.

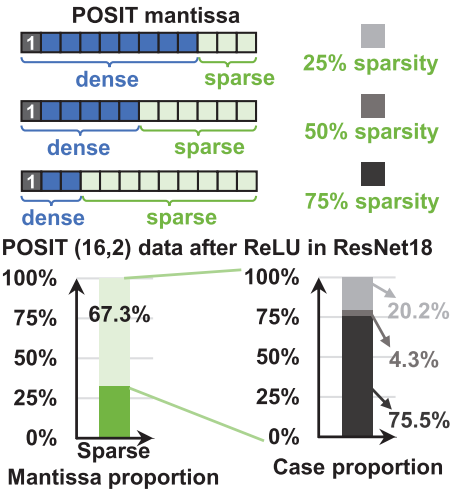


Fig. 6. Bit-level sparsity in the POSIT activation mantissa.

However, the previously developed technologies cannot fully utilize the dynamic precision characteristics of POSIT floating-point numbers. Moreover, although these technologies can reduce the computing power required, they do not significantly improve the computing throughput for CiM macros. In view of this, we need an approximate computing implementation that can flexibly adjust the approximate computing accuracy according to the different mantissa lengths of POSIT data in the CiM architecture while improving the energy efficiency and throughput of the macro.

C. Bitwise Sparsity Optimization

There is pervasive sparsity in large amounts of data in AI workloads. Activation data may have a high sparsity rate after being processed by activation functions such as ReLU. Recent processor-level [20], [21], [22], [23], [24] and macro-level works [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13] based on CiM architecture have proposed efficient optimization methods for data sparsity. These sparse techniques can be classified as blockwise [20], elementwise [23], and bitwise [24] approaches. Of these, the bitwise technique is the most fine-grained. It can exploit the bitwise sparsity in a single non-sparse data sample to improve computing performance (for non-zero-valued data).

As described in Section II-A, the POSIT data format is characterized by dynamically adjusting the length of the mantissa part according to the range of the data. Hence, there is an opportunity for the calculation process to be simplified when the mantissa part of POSIT is shorter. Fig. 6 shows the proportions of activation data with different bitwise sparsity rates for a POSIT precision of (16, 2) on the ResNet18 model. It can be seen that the proportion of data with at least 25% bitwise sparsity accounts for more than 67%. Existing bitwise sparsity optimization techniques aim to minimize the power consumption of the SA in the CiM macro or to effectively reduce the number of input cycles for the bit-serial macro. However, these sparsity technologies cannot achieve joint improvements in energy efficiency and throughput in response to the changing mantissa length characteristic of POSIT numbers in a computing mode where multiple activated data bits are calculated in parallel in a single cycle. We therefore need to develop a bitwise sparsity optimization process that is suitable for bit-parallel calculation and for the POSIT CiM macro.

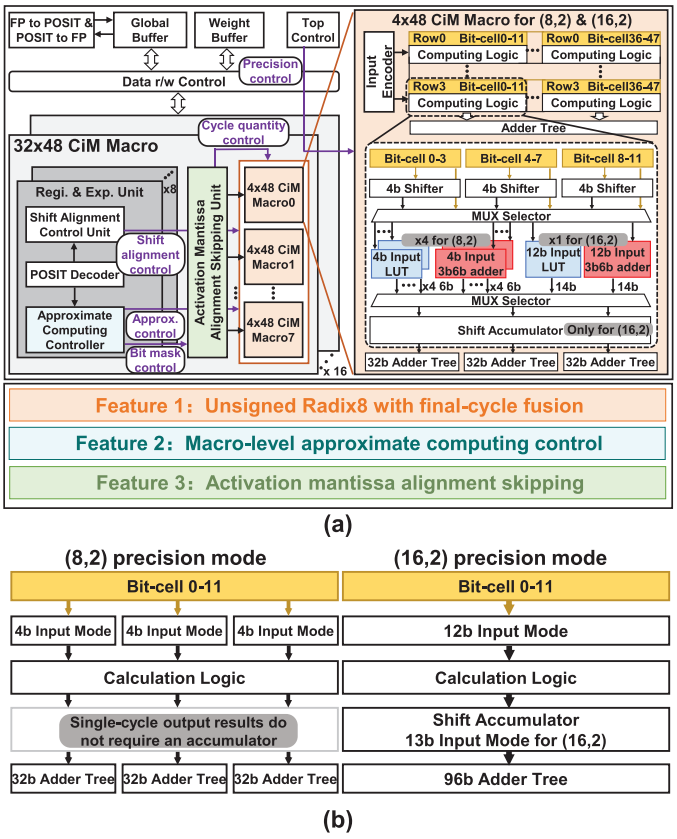


Fig. 7. (a) Proposed top-level CiM macro architecture with three features. (b) (8, 2) and (16, 2) precision modes.

III. Overall Architecture of the Proposed POSIT-Approximate-Calculation-Based CiM Macro

Fig. 7(a) illustrates the overall structure of our chip. It mainly contains 16 32 × 48 CiM macros, a global buffer and weight buffer, a top control, a data R/W controller, and

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:57 UTC from IEEE Xplore. Restrictions apply.

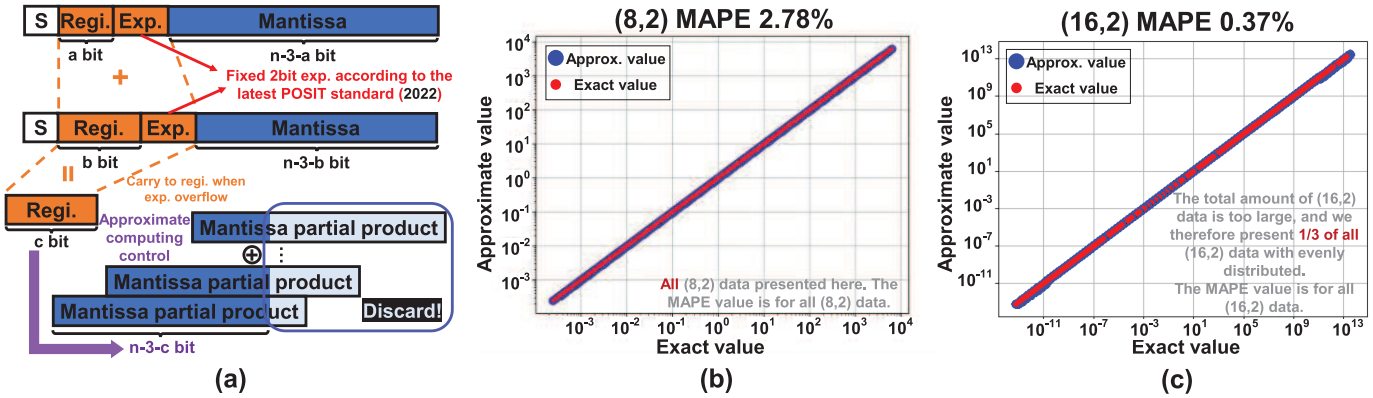


Fig. 8. (a) Principle of the POSIT floating-point approximate calculation. (b) MAPE results for POSIT (8, 2). (c) MAPE results for POSIT (16, 2).

an FP-POSIT transfer unit. Each 32 × 48 macro contains eight regi. and exp. units, one AMAS unit, and eight 4 × 48 macros. Each regi. and exp. unit includes a shift alignment control unit, a POSIT decoder, and an approximate computing controller. The shift alignment control unit is designed to generate the alignment control signal, and the approximate computing controller is devised to output the approximate computing cycle control signal to the AMAS unit and the bit mask signal to the adder tree and shift accumulator. The AMAS unit gives the final control signal for the number of approximate computing cycles used to control the macro. The normalization operation of POSIT floating-point numbers is carried out in the FP-POSIT transfer unit. A single 4 × 48 macro can flexibly support both the (8, 2) and (16, 2) precision modes according to the top control unit.

Fig. 7(b) depicts the detailed configuration principle for the (8, 2) and (16, 2) modes. For (8, 2), each shifter and adder tree only needs to process four columns of bit-cells. Due to the proposed FCF technology, (8,2) only needs one calculation cycle, and the accumulator will be disabled in (8, 2). For (16, 2), three 4-bit input shifters are spliced into a 12-bit shifter. Three 32-bit adder trees are also spliced into a 96-bit adder tree, and the accumulator is enabled.

IV. Unsigned Radix8 Calculation Mode With Final-Cycle Reuse Technology

A. Approximate Calculation Principle

The principle of the POSIT floating-point approximate calculation was first proposed by Zhang and Ko [18]. As shown in Fig. 8(a), before calculating the mantissa multiplication of POSIT, the length of the mantissa part of the multiplication result in POSIT data is first calculated. When performing partial product (PP) addition, the remaining part of the mantissa is dynamically discarded according to the length of the result mantissa to reduce the computational power of the mantissa multiplication. Zhang and Ko [18] proposed a digital multiplier that supports the above principle and used the bit mask signal to control which bits of the PP are discarded in the mantissa multiplication. A good compromise between the computational hardware cost and the loss of algorithm accuracy was achieved, but this digital multiplier cannot be used in the CiM architecture.

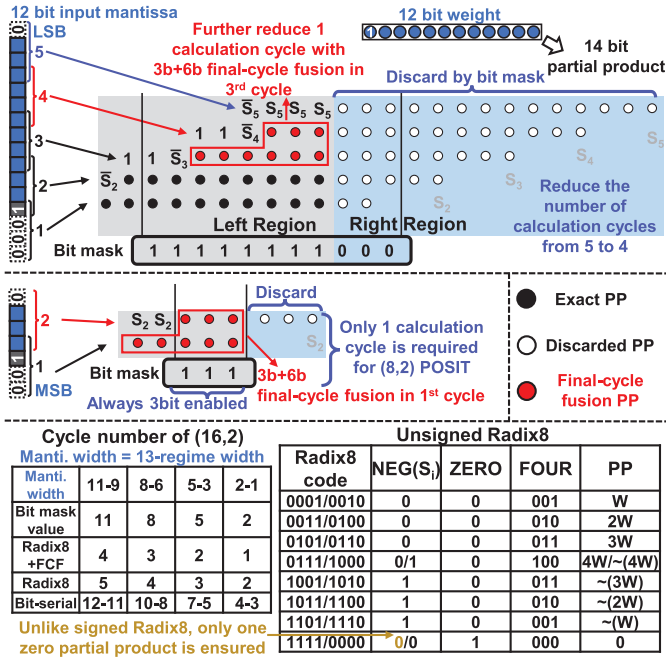
TABLE I
Overall Evaluation of Different NN Models

Model	ResNet18		ViT-B	
Dataset	Cifar-100, Top-1	ImageNet1k, Top-1	ImageNet1k, Top-1	
Activation	POSIT (8,2)	POSIT (16,2)	POSIT (8,2)	POSIT (16,2)
Weight	POSIT (8,2)	POSIT (16,2)	POSIT (8,2)	POSIT (16,2)
Baseline accuracy[1]	75.31%	69.76%	80.25%	
Chip accuracy[2]	74.97% (0.34% loss)	69.69% (0.09% loss)	79.87% (0.38% loss)	80.08% (0.13% loss)
Energy saving[3]	2.6x	5.3x	2.3x	4.8x

1. Baseline accuracy in FP32.
2. Chip tested at 0.55 V, 70 MHz.
3. Baseline is bit-serial POSIT CiM w/o approximate calculation, MACC and AMAS.

For this reason, we first improve the approximate computing principle presented in [18] to make it suitable for the CiM architecture. We then propose a digital CiM macro that supports this new POSIT approximate calculation principle for multistep multiplication calculations suitable for CiM. Fig. 8(b) and (c) shows the MAPE for our improved POSIT approximate calculation principle at two different precisions, (8, 2) and (16, 2), with values of 2.78% and 0.37%, respectively. Fig. 8(b) shows all the data for (8, 2). Since the total data volume for (16, 2) is too large, we only show one-third of it in Fig. 8(c). The MAPE values of both POSIT precisions include the impact of using two error correction techniques. We will introduce the principles of the two error correction techniques in detail in Section VII-B. The only source of the MAPE in this work is the approximate calculation principle. It is this approximate calculation principle that causes part of the mantissa to be discarded during the calculation, which ultimately leads to the MAPE and algorithm accuracy loss. Herein, the Unsigned Radix8 + FCF calculation circuit only fully and accurately implements the approximate calculation. This calculation circuit does not affect the MAPE data in Fig. 8(b) and (c) and the algorithm accuracy in Table I.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:57 UTC from IEEE Xplore. Restrictions apply.



Manti. width	11-9	8-6	5-3	2-1
Bit mask value	11	8	5	2
Radix8 +FCF	4	3	2	1
Radix8	5	4	3	2
Bit-serial	12-11	10-8	7-5	4-3

Radix8 code	NEG(S_i)	ZERO	FOUR	PP
0001/0010	0	0	001	W
0011/0100	0	0	010	2W
0101/0110	0	0	011	3W
0111/1000	0/1	0	100	4W/~(4W)
1001/1010	1	0	011	~(3W)
1011/1100	1	0	010	~(2W)
1101/1110	1	0	001	~(W)
1111/0000	0/0	1	000	0

Fig. 9. Principle of the improved POSIT (16, 2) and (8, 2) approximate calculations based on unsigned Radix8 for CiM architecture.

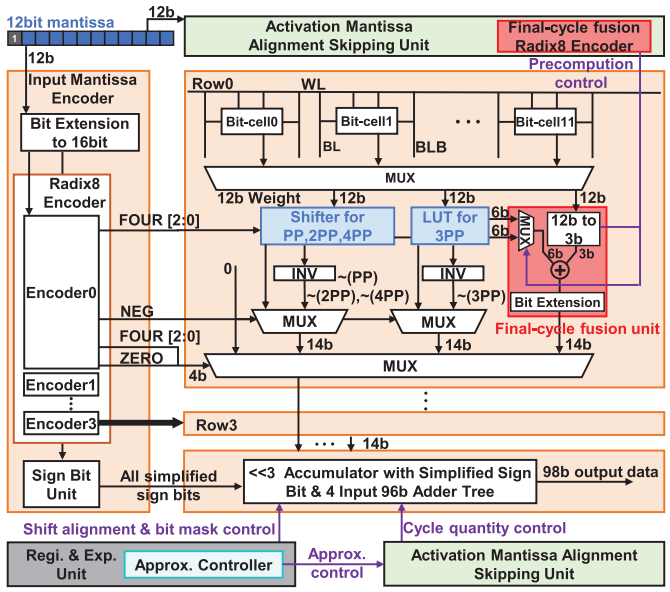


Fig. 10. 4 × 48 CiM macro for the (16, 2) POSIT approximate calculation.

B. POSIT Approximate Calculation in CiM Macro

Fig. 9 shows the details of the approximate calculation of the mantissa multiplication in POSIT using our unsigned Radix8 plus FCF method. For (16, 2) precision, five calculation cycles are required. In the example presented in Fig. 9, the right region is discarded so that the fifth PP at the top does not need to be generated. This means that the mantissa multiplication that originally required five calculation cycles now only requires four. We found that the sum operation of the last two PPs in the left region, after being discarded, could be implemented with only a 3 bit + 6 bit adder. For this reason, we use an FCF unit to implement the sum operation of the last two PPs in the third calculation cycle, further reducing the cycle by one calculation. Finally, the mantissa multiplication is realized within three calculation cycles. For the calculation for (8, 2), our proposed CiM macro only needs one calculation cycle, as the FCF technique is used.

The sign bit of each PP is represented by Si, and its value is equal to NEG in the unsigned Radix8 table, as shown below in Fig. 9. The sign bit of each PP can be directly added in the adder tree using a sign bit unit, according to the simplified form shown in Fig. 9 to illustrate the principle of POSIT (16, 2) and (8, 2) approximate calculations. In Fig. 9, the cycle number for (16, 2) in the table gives the number of calculation cycles and bit mask values for three different approximate calculation implementation methods. The bit mask value refers to the number of 1 s in the bit mask signal, from left to right. As can be seen from the table, our improved approximate calculation method is based on intervals of 3 bits rather than the original interval of 1 bit in the bit mask implementation in [18]. In the improved method, our bit mask only has four cases: 2, 5, 8, and 11. This approach was chosen because it is a better fit for the multiplication calculation principle of Radix8, as the shift interval of each PP in Radix8 is 3 bits. At the same time, our improved approximate calculation principle can retain more remaining bits in PP in some cases. For example, the mantissa width of the result data is 3 or 4, but the bit mask is five. Therefore, this 3-bit interval improvement can also help reduce the errors caused by approximate calculations. Our FCF method can always eliminate one extra calculation cycle compared to Radix8.

Fig. 10 outlines our proposed CiM macro for (16, 2) precision approximate calculations. The input mantissa encoder generates a control signal for Radix8 PPs. The regi. and exp. unit, approx. unit, and AMAS unit are responsible for giving control signals for the approximate calculation. The calculation logic circuit in the CiM macro generates the corresponding PP according to the control circuit. The PP of each cycle is first accumulated in the shift accumulator, and the adder tree is then used to perform the addition operation of the accumulation results from different rows under the control of the alignment signal.

C. Performance Analysis and Comparison

Fig. 11 shows a comparison of the MAC power and throughput of our proposed computing circuit with the Radix8 and bit-serial solutions in the approximate computing mode. We first evaluate the power and throughput of our scheme under all different result mantissa bit width scenarios. For calculations with (16, 2) precision, the proposed Radix8 + FCF solution can achieve lower computing power and higher throughput at any mantissa width. This is because our solution can reduce the number of computing cycles as much as possible while minimizing the additional circuit cost. The proposed FCF technology also eliminates one computing cycle.

To assess the performance gap between approximate and non-approximate calculations, we show the overall performance improvement results of our solution in ResNet18 on

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:57 UTC from IEEE Xplore. Restrictions apply.

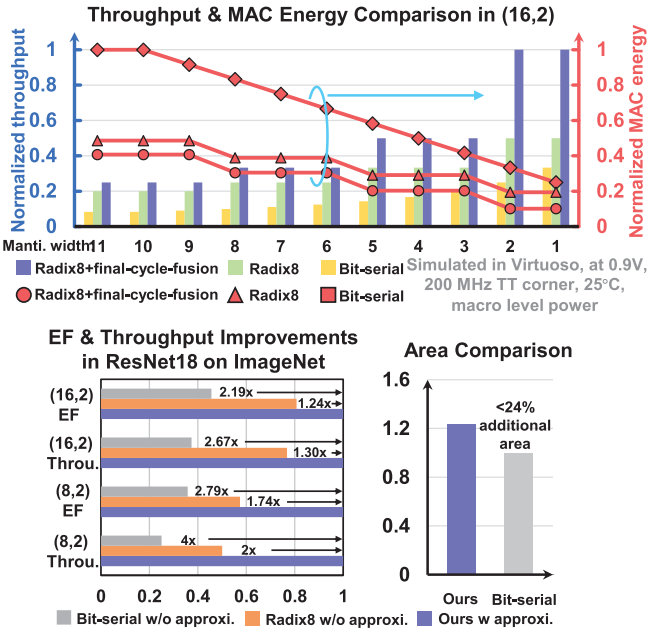


Fig. 11. Performance comparison between the proposed Radix8 + FCF scheme and the existing Radix8 and bit-serial solutions.

the ImageNet dataset under (8, 2) and (16, 2) precision in Fig. 11. The Radix8 and bit-serial solutions do not have an optimizing effect on the approximate calculation. It can be seen that at (16, 2) precision, the energy efficiency can be improved by 1.24–2.19×, while the throughput can be improved by 1.3–2.67×. At (8, 2) precision, the energy efficiency can be improved by 1.74–2.79×, and the throughput can be improved by 2–4×. The throughput improvement effect for POSIT (8, 2) is up to 2–4×, as Radix8 and bit-serial require two and four computing cycles, respectively, in the non-approximate calculation mode, while our Radix8 + FCF solution only requires one computing cycle. The additional area introduced by our proposed solution does not exceed 24%.

V. Macro-Level Approximate Computing Control

A. Three Macro-Level Approximate Computing Strategies

As described in Challenge 2 in Section II, the number of cycles for each multiplication in a CiM macro is randomly distributed for (16, 2) precision. However, due to the rigid structure and fixed data-reuse feature of CiM, it is difficult to flexibly skip all calculation cycles caused by discarded mantissa bits in a single macro. This means that when the calculation with fewer cycles in a single macro has been completed, the part with more cycles still requires calculation cycles. Hence, before the new activation data can be input into the macro for calculation, the model must wait until all data calculations are completed. This will reduce the computing utilization of the macro and lead to a significant waste of computing resources. To solve this problem, we tried three different macro-level control strategies to achieve a good tradeoff between the error of approximate calculation and computing performance.

As shown in Fig. 12, there are three possible strategies to avoid wasting computing resources in a single macro. We first

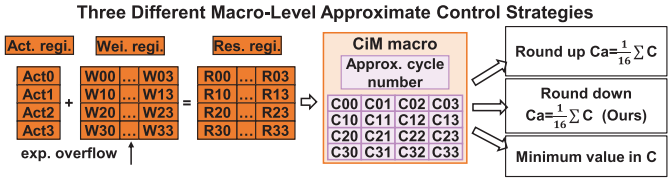


Fig. 12. Three different macro-level approximate calculation strategies.

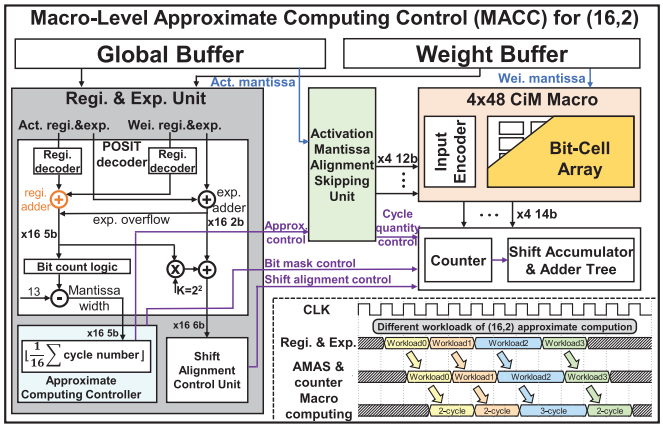


Fig. 13. Circuit implementation details of the macro-level approximate computing control (MACC) for (16, 2).

calculate the 16 cycle numbers required for 16 multiplication calculations in a single 4 × 48 macro. We can then choose from three control strategies: rounding up, rounding down, and taking the minimum value. The contributions of these three strategies are as follows.

1) They can integrate different numbers of calculation cycles in a CIM macro into one number of calculation cycles.
2) The three strategies can bring different acceleration effects and approximate calculation errors. This means that the strategy can be selected according to the actual calculation scenario.
3) The three strategies fully explore the tradeoff design space between approximate calculation error and acceleration effect. The final solution implemented here is to round down. In the following sections, we will introduce the circuit design of MACC and the reason for choosing the rounding-down strategy.

B. Implementation Details of Our MACC Circuit

Fig. 13 shows the circuit implementation details of the proposed MACC. First, the regime and exponent data of the 20 operands (four activation data and 16 weight data) are sent to the regi. and exp. unit from the global buffer and weight buffer. The regi. and exp. unit is responsible for calculating the values of the regime and exponent using the decoder. These 20 values can be used to calculate the 16 regime values of the 16 multiplication results according to the multiplication calculation rules of POSIT floating-point numbers. The bit count logic in Fig. 13 uses these 16 regime values to calculate the bit width of the regime part in the multiplication result

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:57 UTC from IEEE Xplore. Restrictions apply.

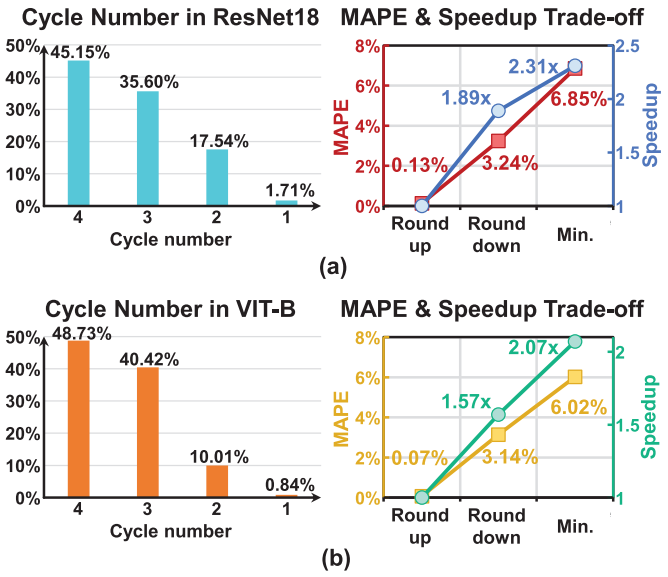


Fig. 14. (a) Distribution of the cycle number and MAPE and speed improvement in ResNet18. (b) Distribution of the cycle number and MAPE and speed improvement in VIT-B.

data. We then use 13 to subtract the width of each regime to get the width of the mantissa for each multiplication result data. A value of 13 is used as the minuend because, in the POSIT data of the multiplication result, 13 is the sum of the widths of the regime and mantissa parts (except for the 1-bit sign bit and the 2-bit exponent bit). The last 16 mantissa width values will be sent to the approximate computing control module to calculate the average value and round down the result. Finally, the MACC processes the 16 calculation cycle numbers for a single 4 $\times$ 48 macro into one common calculation cycle number for (16, 2) precision.

Fig. 13 also shows three control signals given by the regi. and exp. unit. A total of three control signals need to be generated, as marked with purple signal lines and text descriptions in the figure, namely, approximate control, bit mask control, and shift alignment control. The approximate control signal is used to control the number of computing cycles of a single 4 $\times$ 48 macro. This signal also needs to be further processed by the AMAS unit to generate the final macro control signal. Since the approximate computing control module in the regi. and exp. unit has determined the number of computing cycles caused by the approximate computing principle. According to the table in Fig. 9, we can get the bit mask control signal used to control the entire macro. All multiplication calculations in this macro use the same bit mask control signal in the shift accumulator and adder tree. The final shift alignment control is used to align the mantissa parts of different floating-point numbers to complete the addition operation. Fig. 13 shows the timing diagram for the MACC when processing different workloads.

C. Performance Evaluation

Fig. 14 shows the distribution of the cycle numbers in ResNet18 and VIT-B on the Imagenet dataset and the performance improvement effects from the three different MACC strategies. As can be seen from Fig. 14, in more than 50% of the multiplications, the number of calculation cycles is less than four. This means that more than half of the approximate multiplications present opportunities for optimization. We also tested the MAPE values and speedup effects of the three different macro-level control strategies. The calculation of all MAPE values takes into account the impact of introducing two error correction techniques.

Because the implementation circuits of the three control strategies are not complex, the main factors we consider when making choices are the approximate calculation error and the improvement of calculation performance. Fig. 14 shows that the rounding-up operation can improve the MAPE from 0.37% to 0.07%-0.13% in different neural network (NN) models. However, rounding-up operation is also impossible to improve energy efficiency and throughput from approximate calculations. Although taking the minimum value can achieve the best acceleration effect, the MAPE value will deteriorate to 6.02%–6.85%. We therefore chose the strategy of taking the average value and rounding it down to achieve a good tradeoff. This gives an acceleration effect of 1.57×−1.89×, and the MAPE value deteriorates by only 3.14%–3.24%. The extra circuit area and circuit power introduced by the MACC and AMAS are only 2.78% and 3.79%, respectively.

VI. Activation Mantissa Alignment Skipping

A. Sparse Bit Analysis

Sparse bits are randomly distributed over the entire activation data of the algorithm. The multiplication calculation principle we implemented is based on the Radix8 multiplier, which can handle a 4-bit activation mantissa in each cycle; hence, the sparse bits in the activation mantissa can only be skipped by our calculation circuit to improve the computing energy efficiency and throughput when they have certain distribution characteristics. Fig. 15 shows the results of an analysis of the sparse bits in the POSIT (16, 2) activation mantissa. The value of Ci in the figure represents the number of valid input cycles in each case. The impact of FCF is also taken into account when calculating the number of valid input cycles. We can divide the bit sparsity in POSIT into three cases, each of which will result in one to three valid input cycles. As shown in Fig. 15, these three cases account for more than 67% of the ResNet18 algorithm. Some of these sparse data are caused by the short length of the POSIT mantissa, and the remainder are caused by the sparse bits of the POSIT mantissa itself. Due to the use of the ReLU activation function, the case with a valid input cycle equal to one accounts for the highest proportion among all sparse mantissa data. The variable Ca in the figure represents the final number of calculation cycles caused by the approximate calculation. The value of Ca is given by the approximate control signal mentioned in Section V-B. In the following, we will explain in detail how these two values of Ca and Ci enable the calculation cycle control of a single macro to be realized.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:57 UTC from IEEE Xplore. Restrictions apply.

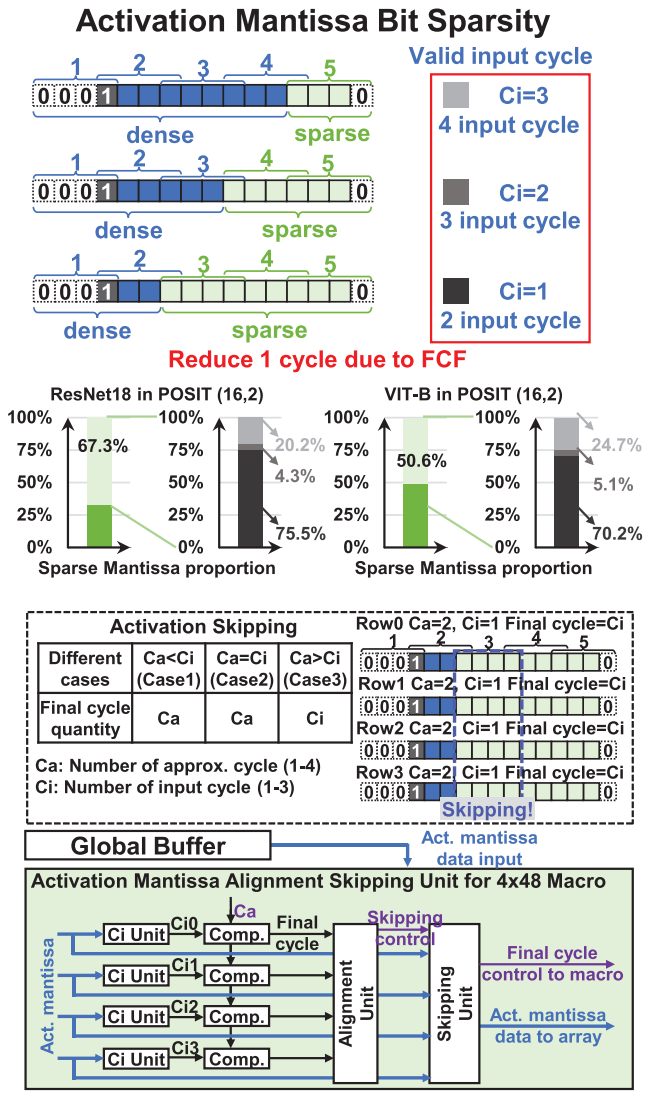


Fig. 15. Sparse mantissa proportion and principle of the AMAS process and circuit implementation for (16, 2).

B. Ca/Ci Joint Skipping

The approximate computing controller unit obtains the number of computing cycles Ca, for a single macro based on the approximate computing principle. According to the sparse distribution of the input activation mantissas, we obtain the effective input period Ci. There are three cases for the relative sizes of the two values Ca/Ci, as shown in the table in Fig. 15. The table shows which of the two values determines the final number of calculation cycles of the macro in the three cases. When Ca = Ci, the control effect of the two is the same. In the other two cases, the smaller value of Ca and Ci is selected. This approach was used because, in our design, the principle of approximate computing is used to control the tradeoff between computational error and energy efficiency for each calculation. The number of calculation cycles for a single macro needs to be sufficient to meet the needs of the approximate calculation first, and the issue of whether meaningless sparse bits can be skipped is then considered.

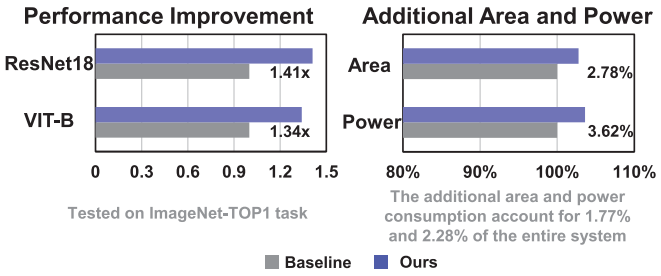


Fig. 16. Performance results for AMAS and additional area/power from MACC and AMAS.

Fig. 15 also gives an example of how Ca/Ci determines the final number of calculation cycles. It can be seen that for a single 4 × 48 macro, four rows of parallel input can be supported. In the figure, Ca = 2 and Ci = 1 for Rows 0–3, so the final cycle of each row is Ci. Since the final number of calculation cycles for the four rows is the same, they can be skipped at the same time, thereby reducing the number of calculation cycles for one macro. The approximate calculation approach reduces the calculation, which originally required four cycles (considering FCF), to only two cycles, and the sparse bit distribution scheme of the activated mantissa then results in only one calculation cycle. The microarchitecture details of this sparse control are given at the bottom of Fig. 15. First, the input mantissa from the global buffer is sent to the Ci unit of each row to calculate its Ci value, and it is then compared with Ca in the comparator unit. Finally, the final-cycle value of a single 4 × 48 macro is obtained. The alignment unit and skipping unit determine whether sparse calculations in these four rows can be skipped at the same time. The skipping unit will send the final calculation cycle control signal and the mantissa data to be input to the macro.

C. Performance Evaluation

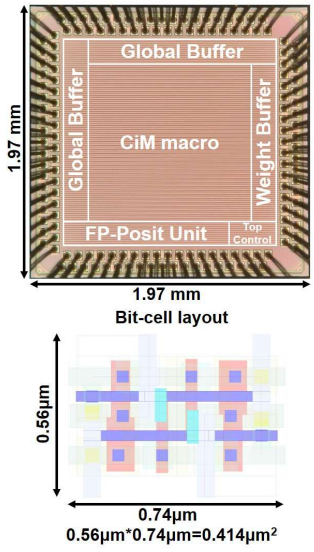
Fig. 16 shows the results of a performance and cost analysis of the MACC and AMAS modules. Using ResNet18 and VIT-B on the ImageNet-Top1 task, we tested the acceleration effect of the proposed AMAS scheme. Based on the approximate calculation acceleration, the AMAS module helps to reduce the number of calculation cycles of the CiM macro further by leveraging the sparsity in the activation mantissa data and achieves an additional 1.34× − 1.41× acceleration in different NN models. Our MACC and AMAS technologies only introduce an extra 2.78% to the circuit area in total, and the power consumption of this additional circuit is only 3.62% of the entire macro power consumption. The additional area and power consumption account for 1.77% and 2.28% of the entire system, respectively.

VII. Measurement Results

A. Fabricated Chip and Measurement Platform

Fig. 17 shows a chip die photograph, a performance summary table, and the SRAM bit-cell layout. The proposed POSIT-approximate-calculation-based macro was fabricated using 28-nm complementary metal oxide semiconductor

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:57 UTC from IEEE Xplore. Restrictions apply.



CMOS	28nm
Chip Area	1.97 x 1.97mm²
CiM/Buffer Size	24Kb/17KB
Bit-Cell Area	0.414μm²
Macro Area	0.127mm²x16
Supply Voltage	0.55-1.1V
Clock Frequency	70-280MHz
Power	9.7-85.6mW
Supported Precision	POSIT(8,2) (Act/W) POSIT(16,2) (Act/W)
Peak Performance*1*2	3.44TFLOPS @POSIT(8,2) 0.89TFLOPS @POSIT(16,2)
Macro Energy Efficiency*1*3	40.1-88.3TFLOPS/W @POSIT(8,2) 10.3-22.8 TFLOPS/W @POSIT(16,2)
Macro Area Efficiency (280MHz with 3 features)	1.68TFLOPS/mm² @POSIT(8,2) 0.43TFLOPS/mm² @POSIT(16,2)

*1 1 operation (OP) = 1 multiplication/addition.
*2. Maximum throughput tested at 1.1V 280MHz. MACC & AMAS are included. The performance improvement of MACC is 1.89x. The performance improvement of AMAS is 1.22-1.65x according to the test data, and the value 1.65 is taken here to calculate the peak performance.
*3. Off-chip/IO power and latency is not included. Highest efficiency point at 0.55 V, 70MHz.

Fig. 17. Chip photograph and performance summary.

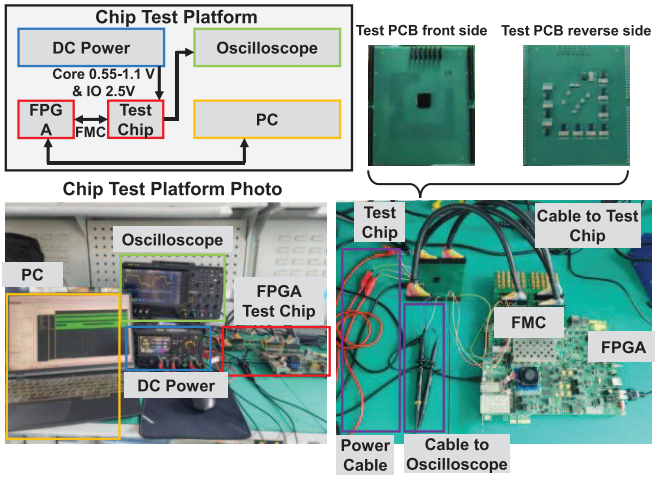


Fig. 18. Test platform for our CiM macro chip.

(CMOS) technology and occupied a total area of 1.97 × 1.97 = 3.88 mm^2. The test platform for the CiM macro included a host PC, an Xilinx ZCU-106 field-programmable gate array (FPGA) board, a test printed circuit board (PCB), two FPGA mezzanine connector (FMC) boards, a dc power supply, and an oscilloscope. Fig. 18 shows the front side of the test PCB with our CiM chip and the pins for testing. On the reverse side of the PCB are the capacitor and resistor components and the level shifter chip needed for testing. The oscilloscope was used to track the working status signal of the chip. All the data from the test dataset and the weight data for the network were preloaded from the FPGA DRAM into the on-chip global buffer and weight buffer through the FMC board and the cable connecting to it. The FPGA input test signals to the test chip and performs data transmission. The calculation result was collected by the FPGA and returned to the host PC for verification purposes.

The core operating voltage, IO operating voltage, and level shifter chip operating voltage on the PCB were supplied by a

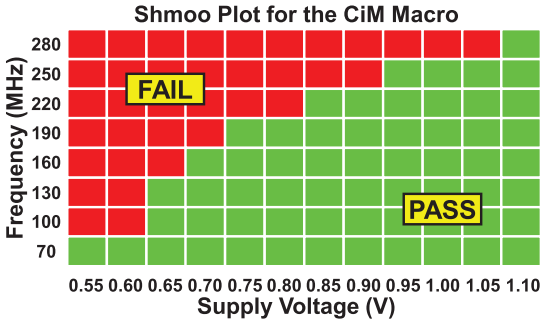


Fig. 19. Voltage and frequency scaling.

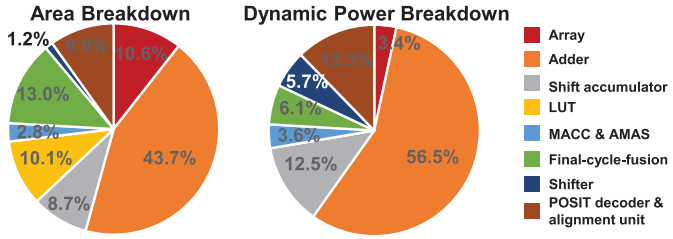


Fig. 20. Area and power breakdown for our CiM macro.

dc voltage source. The 1.8-V test signal from the FPGA was converted into a 2.5-V voltage by the level shifter chip on the PCB test board and input into the chip. The chip output signal was also converted to 1.8 V by the level shifter chip for reception by the FPGA-based logic analyzer. The power consumption of the chip I/O and FPGA DRAM was excluded. The peak macro energy efficiency reached 88.3 TFLOPS/W @ POSIT8 and 22.8 TFLOPS/W @ POSIT16 with three features at 0.55 V and 70 MHz.

As shown in Fig. 19, the core voltage of our prototype could be adjusted between 0.55 and 1.1 V when setting the I/O voltage of 2.5 V, and the working frequency ranged from 70 to 280 MHz. Fig. 20 illustrates the area and power (power of bit-cell array is not included) breakdown of the POSIT-approximate-calculation-based CiM macro. The LUT circuit required an additional area of 10.1% in order to reduce the power consumption needed to generate the +3/-3 weight PP. The circuit area for the LUT and FCF accounts for 23.1% of the entire macro. Compared with the bit-serial solution, this 23.1% is the main additional area. In terms of power consumption, the FCF technology proposed here only accounts for 6.1% of the power consumption, but it can help to further reduce the entire macro by one calculation cycle.

B. Test Benchmark and Error Correction for Approximate Calculations

Fig. 21 shows the error correction details for the approximate calculation. In our improved approximate calculation method, the main source of error is the discarded mantissa bit. However, we found two techniques that could help reduce the overall approximate error.

1) We found that in the traditional Radix8 method used for INT data calculation in the CiM macro, when the Radix8code was “1111” and “0000,” a PP with a value

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:57 UTC from IEEE Xplore. Restrictions apply.

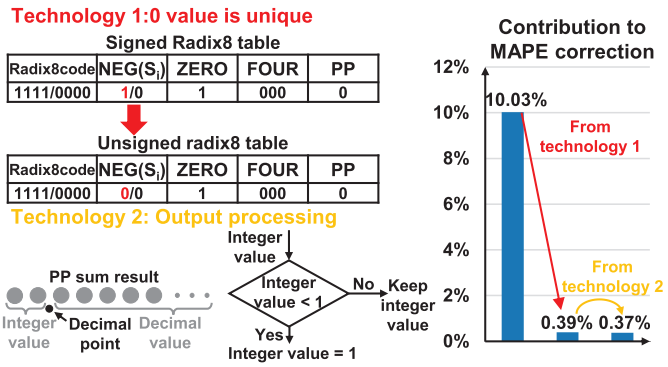


Fig. 21. Error sources and error correction for POSIT (16, 2) approximate calculation.

of zero was generated. However, when the Radix8code is "1111," the value of NEG is "1" in the traditional method. We found that this "1" value caused an incorrect carry in the INT part, which caused the value of the INT part to exceed three. However, in the mantissa multiplication of floating-point multiplication, the maximum value of the INT part can only be three. We tried changing the NEG value when the Radix8code was "1111" to "0" and found that this small modification greatly reduced the approximate calculation error for POSIT (16, 2). This reduced the overall MAPE value at a precision of (16, 2) from 10.03% to 0.39%. Due to this modification, the two cases of Radix8code "1111" and "0000" are combined into one case, and we can describe this technology with the phrase "zero value is unique."

2) We found that the discarded mantissa of the approximate calculation may cause the value of the INT part to be less than one because the discarded mantissa bits may result in the disappearance of the carry. Obviously, this is incorrect in the multiplication calculation of floating-point numbers. For this reason, we added judgment logic to the calculation logic circuit for POSIT. When the INT value is less than one, it is set to one. This output processing technology improves the overall MAPE value for (16, 2) from 0.39% to 0.37%.

Finally, these two error correction technologies reduced the MAPE value for (16, 2) to 0.37%. We found that these two error correction techniques had almost no impact on the MAPE value for (8, 2), so only the improvements in the precision for (16, 2) are given in Fig. 21. Fig. 22 shows the distribution characteristics of MAPE values for (8, 2) and (16, 2) in different data intervals after the introduction of the two error correction techniques. We can see that the calculation error of (8, 2) is very evenly distributed over data intervals of different orders of magnitude and is consistently around 2.78%. In the POSIT (16, 2) precision in Fig. 22, we use four different colors to distinguish different numbers of computation cycles and give the MAPE value for each of these four cases. The MAPE values for (16, 2) are less than 0.37% between 10^{-8} and 10^{6}. The MAPE values significantly deteriorate for extremely large and extremely small data intervals, but due to the normal distribution characteristics of the data in the AI algorithm [15],

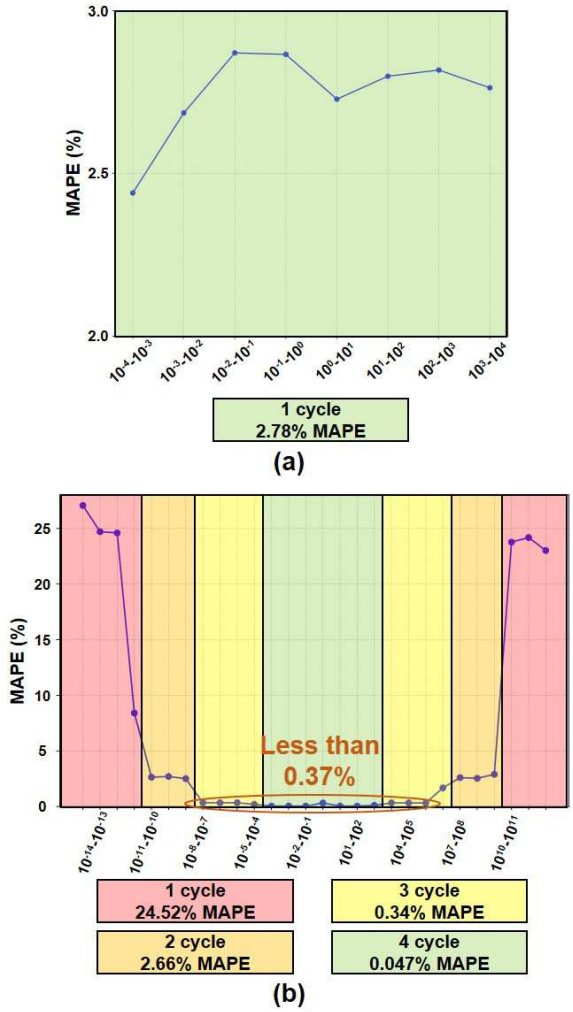


Fig. 22. Distribution of MAPE values at a precision of (a) (8, 2) and (b) (16, 2).

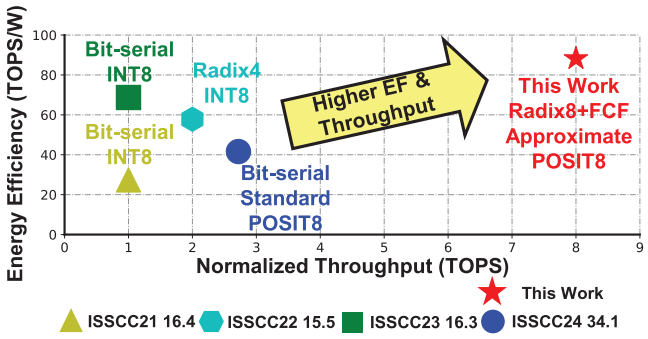


Fig. 23. Comparison between algorithm accuracy, energy efficiency, and normalized throughput.

[27], [28], our approximate calculation scheme does not have a significant impact on the accuracy of the algorithm.

To evaluate our POSIT-approximate-calculation-based CiM macro, we trained the ResNet18 and ViT-B algorithms on two different tasks, Cifar-100-Top1 and ImageNet-Top1. Table I shows the performance data for our approximate calculation method and the chip for two different POSIT precisions. In the ResNet18 test, for a POSIT precision of (8, 2), the proposed scheme achieves an accuracy of 74.89% on the inference

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:57 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON WITH STATE-OF-THE-ART DIGITAL CIM-BASED MACROS

	ISSCC 2021 16.4 [3]	ISSCC 2022 15.5 [22]	ISSCC 2023 16.3 [23]	ISSCC 2021 9.4 [17]	ISSCC 2024 34.1 [2]	**This Work**
Architecture	CiM	CiM	CiM	Digital	CiM	**CiM**
CiM macro implementation	Bit-serial	Radix4	Bit-serial	NA	Bit-serial	**Unsigned Radix8+ final-cycle fusion**
Dynamic format	No	No	No	Yes	Yes	**Yes**
Approximate calculation	No	No	No	No	No	**Yes**
CMOS technology (nm)	22	28	28	28	28	**28**
Supply voltage (V)	0.72	0.6–1.0	0.397–0.9	0.6–0.9	0.55–1.0	**0.55–1.0**
Die area (mm^2)	0.202 (macro)	6.96	4.54	3.8	1.41	**3.88**
Frequency (MHz)	500	50–220	10–400	104–288	78–419	**70–280**
Supported precision	INT4/INT8	INT8/INT16 BF16/FP32	INT4/INT8 BF16/FP16	8b/16b/32b CUSTOM POSIT	POSIT(8,1)(8,2) POSIT(16,2)	**POSIT (8,2) POSIT (16,2)**
CiM cycle number POSIT (8,2)/(16,2)	NA	NA	NA	NA	2-4/8-12[2]	**1/1-4**
Macro sparsity support	No	No	No	No	No	**Act. mantissa bitwise sparsity**
Peak macro energy efficiency for traditional precision[1] (TOPS/W, TFLOPS/W)	22.3@INT8	36.5@INT8 3.7@FP32	68.7@INT8 17.2@FP16	NA	NA	**NA**
Peak macro energy efficiency for POSIT[1] (TFLOPS/W)	NA	NA	NA	0.248@POSIT8 0.121@POSIT16 (System)	41.6@POSIT8[3] 19.2@POSIT16[3]	**88.3@POSIT8[4] 22.8@POSIT16[4]**

(1) All energy efficiency data are from dense test data to enable a fair comparison; energy efficiency improvement from element-level sparsity is not taken into account. Off-chip/IO power and latency are not included. 1 operation (OP) = 1 multiplication/addition. (2) Speed up by feature2 is enabled. (3) The previously reported energy efficiency values (83.32@PSOIT8 and 38.37@POSIT16) under 50% input sparsity were multiplied by two. The energy efficiency values do not account for the improvement resulting from input sparsity. (4) Highest efficiency point at 0.55 V, 70MHz. MACC & AMAS are enabled. Performance improvement from sparsity only includes sparse bits in the mantissa.

task of the Cifar-100 dataset, a loss of only 0.34% compared to the baseline accuracy of FP32. For a POSIT precision of (16, 2), an accuracy loss of only 0.09% is achieved on the ImageNet training task compared to FP32. In the ViT-B test, the accuracy is 0.38% and 0.13% in POSIT (8, 2) and POSIT (16, 2), respectively. This proves that the accuracy loss caused by our improved approximate calculation is very small. Our chip also achieved energy savings of 2.3–5.3× at the two POSIT precision tests, compared with the bit-serial solution without approximate calculation, MACC, and AMAS. The performance improvement for (8, 2) mainly arises from the improved approximate calculation principle and unsigned Radix8 + FCF technology. For (16, 2), the performance also benefits from the optimization effects of MACC and AMAS, so the improvement is better than for (8, 2). This also proves that the approximate calculation principle and hardware implementation proposed here can achieve a better performance improvement than the baseline when the precision of POSIT is extended to a higher mantissa precision [such as (32, 2)].

C. Comparison With the State of the Art

Table II shows a comparison between this work and state-of-the-art CiM macros. As few computing hardware schemes support POSIT floating-point numbers, there are only two works related to POSIT calculation in this comparison table [2], [17]. Due to the particular characteristics of POSIT, it is difficult to compare the energy efficiency data of our chip with INT/FP type data. It should be noted that under POSIT (8, 2) and (16, 2) precision, the maximum lengths of the mantissa part are 4 and 12 bits, respectively. The mantissa lengths of FP16 and BF16 are 11 and 8 bits, respectively (including hidden bits). To ensure a fair comparison, we uniformly converted all energy efficiency data in the table into test results under dense test data. This means that the performance improvement offered by all element-wise sparsity is not taken into account. Since our CiM macro uses AMAS technology as an optimization method for the changing characteristics of the mantissa length and bit sparsity of POSIT, the energy efficiency data in this work include the effect of bitwise sparsity.

The study in [3] was the first work on an SRAM-based digital CiM macro. LUT technology was used to optimize the power consumption and area of the first-stage adder [4]. The scheme in [22] was the first SRAM digital CiM macro to support floating-point calculations. Its use of pre-aligned operations significantly reduced the power consumption of floating-point calculations in the CiM macro. Yue et al. [21]

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:57 UTC from IEEE Xplore. Restrictions apply.

proposed an interpolating bits technology that significantly reduced the number of calculation cycles of the CiM macro for floating-point operations by interpolating sign bits and zero bits. Shah et al. [17] were the first to design a digital architecture processor for POSIT. Wang et al. [2] first proposed a digital CiM macro supporting POSIT calculations, which used universal optimization methods to reduce the power consumption of input processing, mantissa multiplication, and macro internal addition calculation. In the present work, we report a POSIT-approximate-calculation-based CiM macro for the first time. In the improved approximate calculation method, an unsigned Radix8 + FCF method is applied to significantly reduce the number of calculation cycles while improving energy efficiency. MACC technology is used to solve the problem of CiM macro's inability to flexibly skip calculation cycles and find a suitable tradeoff between the acceleration effect and calculation error. In view of the bitwise sparsity of activation data and the short mantissa data of POSIT in AI algorithms, the AMAS method was proposed to further reduce sparse calculation costs based on approximate calculation optimization. As shown in Table II, our CiM macro achieved a peak macro-level energy efficiency of 88.3 TFLOPS/W @ POSIT8 and 22.8 TFLOPS/W @ POSIT16, values that were 1.18–2.12× higher than the prior POSIT CiM macro scheme.

VIII. Conclusion and Discussion

To further improve the performance of CiM architecture in calculating POSIT floating-point numbers, this article has proposed a POSIT-approximate-calculation-based CiM macro. Because of the characteristics of POSIT's variable-length mantissa, the mantissa length of the multiplication result data was determined before calculation. Based on this mantissa length, the mantissa part that needs to be accumulated is dynamically reduced to reduce the computational cost of POSIT multiplication. Compared with prior bit-serial/Radix4 support for INT8 and bit-serial-based support for POSIT8, our digital CiM macro, which was designed based on Radix8 + FCF to support approximate POSIT computing, significantly improved throughput while achieving higher computing energy efficiency. The error analysis on different data intervals of the POSIT approximate calculation and the final algorithm test results can prove that the POSIT approximate calculation is a powerful method to further improve its computing efficiency by leveraging the variable-length mantissa feature of POSIT. Hopefully, the techniques and data analysis proposed in this article can further help AI computing hardware improve computing energy efficiency. A 28-nm prototype achieved almost the peak macro energy efficiency of 88.3 TFLOPS/W for POSIT (8, 2) and 22.8 TFLOPS/W for POSIT (16, 2) at 0.55 V and 70 MHz.

References

[1] N. Verma et al., "In-memory computing: Advances and prospects," *IEEE Solid State Circuits Mag.*, vol. 11, no. 3, pp. 43–55, Summer. 2019.

[2] Y. Wang et al., "34.1 a 28nm 83.23TFLOPS/W POSIT-based compute-in-memory macro for high-accuracy AI applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 566–568.

[3] Y.-D. Chih et al., "16.4 an 89TOPS/W and 16.3TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, San Francisco, CA, USA, Feb. 2021, pp. 252–254.

[4] C.-F. Lee et al., "A 12nm 121-TOPS/W 41.6-TOPS/mm^2 all digital full precision SRAM-based Compute-in-Memory with configurable bit-width for AI edge applications," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Honolulu, HI, USA, Jun. 2022, pp. 24–25.

[5] H. Mori et al., "A 4nm 6163-TOPS/W/b *4790-TOPS/mm^2/b* SRAM based digital-computing-in-memory macro supporting bit-width flexibility and simultaneous MAC and weight update," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 132–134, doi: 10.1109/ISSCC42615.2023.10067555.

[6] H. Fujiwara et al., "34.4 a 3nm, 32.5TOPS/W, 55.0TOPS/mm^2 and 3.78Mb/mm2 fully-digital compute-in-memory macro supporting INT12 × INT12 with a parallel-MAC architecture and foundry 6T-SRAM bit cell," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 572–574, doi: 10.1109/ISSCC49657.2024.10454556.

[7] Y. He et al., "7.3 a 28nm 38-to-102-TOPS/W 8b multiply-less approximate digital SRAM compute-in-memory macro for neural-network inference," in *Proc. IEEE Int. Solid- State Circuits Conf. (ISSCC)*, San Francisco, CA, USA, Feb. 2023, pp. 130–132.

[8] P. Chen et al., "7.8 a 22nm delta-sigma computing-in-memory (ΔΣCIM) SRAM macro with near-zero-mean outputs and LSB-first ADCs achieving 21.38TOPS/W for 8b-MAC edge AI processing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 140–142.

[9] W.-S. Khwa et al., "34.2 a 16nm 96Kb integer/floating-point dual-mode-gain-cell-computing-in-memory macro achieving 73.3-163.3TOPS/W and 33.2-91.2TFLOPS/W for AI-edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 568–570.

[10] A. Guo et al., "34.3 a 22nm 64kb lightning-like hybrid computing-in-memory macro with a compressed adder tree and analog-storage quantizers for transformer and CNNs," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 570–572.

[11] T.-H. Wen et al., "34.8 a 22nm 16Mb floating-point ReRAM compute-in-memory macro with 31.2TFLOPS/W for AI edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 580–582.

[12] P.-C. Wu et al., "A 22nm 832Kb hybrid-domain floating-point SRAM in-memory-compute macro with 16.2-70.2TFLOPS/W for high-accuracy AI-edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 126–128.

[13] Y. Yuan et al., "34.6 a 28nm 72.12TFLOPS/W hybrid-domain outer-product based floating-point SRAM computing-in-memory macro with logarithm bit-width residual ADC," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 576–578.

[14] J. L. Gustafson and I. Yonemoto, "Beating floating point at its own game: Posit arithmetic," *Supercomput. Frontiers Innov.*, vol. 4, no. 2, pp. 71–86, 2017.

[15] R. Murillo, A. A. Del Barrio, and G. Botella, "Deep PeNSieve: A deep learning framework based on the posit number system," *Digit. Signal Process.*, vol. 102, Jul. 2020, Art. no. 102762.

[16] G. Raposo, P. Tomás, and N. Roma, "Positnn: Training deep neural networks with mixed low-precision posit," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Toronto, ON, Canada, Jun. 2021, pp. 7908–7912.

[17] N. Shah, L. I. G. Olascoaga, S. Zhao, W. Meert, and M. Verhelst, "9.4 PIU: A 248GOPS/W stream-based processor for irregular probabilistic inference networks using precision-scalable posit arithmetic in 28nm," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, San Francisco, CA, USA, Feb. 2021, pp. 150–152.

[18] H. Zhang and S.-B. Ko, "Efficient approximate posit multipliers for deep learning computation," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 13, no. 1, pp. 201–211, Mar. 2023.

[19] Y. Wang, D. Deng, L. Liu, S. Wei, and S. Yin, "PL-NPU: An energy-efficient edge-device DNN training processor with posit-based logarithm-domain computing," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 10, pp. 4042–4055, Oct. 2022.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:57 UTC from IEEE Xplore. Restrictions apply.

[20] J. Yue et al., "15.2 a 2.75-to-75.9TOPS/W computing-in-memory NN processor supporting set-associate block-wise zero skipping and ping-pong CIM with simultaneous computation and weight updating," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, San Francisco, CA, USA, Feb. 2021, pp. 238–240.
[21] J. Yue et al., "A 28nm 16.9-300TOPS/W computing-in-memory processor supporting floating-point NN inference/training with intensive-CIM sparse-digital architecture," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 1–3.
[22] F. Tu et al., "A 28nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 1–3.
[23] F. Tu et al., "16.4 TensorCIM: A 28nm 3.7nJ/Gather and 8.3TFLOPS/W FP32 digital-CIM tensor processor for MCM-CIM-based beyond-NN acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 254–256.
[24] F. Tu et al., "16.1 MuITCIM: A 28nm *2.24 μJ*/token attention-token-bit hybrid sparse digital CIM-based accelerator for multimodal transformers," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 248–250.
[25] D. Wang, C.-T. Lin, G. K. Chen, P. Knag, R. K. Krishnamurthy, and M. Seok, "DIMC: 2219TOPS/W 2569F2/b digital in-memory computing macro in 28nm based on approximate arithmetic hardware," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, San Francisco, CA, USA, Feb. 2022, pp. 266–268.
[26] H. Wu et al., "A 28-nm 19.9-to-258.5-TOPS/W 8b digital computing-in-memory processor with two-cycle macro featuring winograd-domain convolution and macro-level parallel dual-side sparsity," *IEEE J. Solid-State Circuits*, vol. 60, no. 1, pp. 347–361, Jan. 2025.
[27] Z. Yue et al., "14.4 a 51.6TFLOPs/W full-datapath CIM macro approaching sparsity bound and <2-30 loss for compound AI," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2025, pp. 1–3.
[28] X. Wang et al., "14.3 a 28nm 17.83-to-62.84TFLOPS/W broadcast-alignment floating-point CIM macro with non-two's-complement MAC for CNNs and transformers," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2025, pp. 254–256.
[29] [Online]. Available: https://posithub.org/docs/posit_standard-2.pdf
[30] M.-E. Shih et al., "20.1 NVE: A 3nm 23.2TOPS/W 12b-digital-CIM-based neural engine for high-resolution visual-quality enhancement on smart devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 360–362.



Yong Chen (Senior Member, IEEE) received the B.Eng. degree in electronic and information engineering from the Communication University of China (CUC), Beijing, China, in 2005, and the Ph.D. degree in engineering degree in microelectronics and solid-state electronics from the Institute of Microelectronics of the Chinese Academy of Sciences (IMECAS), Beijing, in 2010.

From 2010 to 2013, he was a Post-Doctoral Researcher with the Institute of Microelectronics, Tsinghua University, Beijing. From 2013 to 2016, he was a Research Fellow with VIRTUS/EEE, Nanyang Technological University, Singapore. Then, he was an Assistant Professor from 2016 to 2022 and an Associate Professor from 2022 to 2024 with the State Key Laboratory of Analog and Mixed-Signal VLSI (AMSV), University of Macau, Macao, China. In 2024, he joined the Department of Electronic Engineering, Tsinghua University, Beijing, where he is currently an Associate Professor. His research interests include integrated circuit designs involving analog/mixed-signal/RF/mm-wave/sub-THz/wireline.

Dr. Chen was a recipient of the "Haixi" (three places across the Straits) postgraduate integrated circuit design competition (Second Prize) in 2009 and a co-recipient of the Best Paper Award from the IEEE Asia Pacific Conference on Circuits and Systems (APCCAS) in 2019, the Best Student Paper Award (Third Place) at the IEEE Radio Frequency Integrated Circuits (RFIC) Symposium in 2021, Macao Science and Technology Invention Award (First Prize) in 2020, Macao Science and Technology Invention Award (Second Prize) in 2022, the Gold Leaf Prize (Top 10% Papers) at the 18th International Conference on Ph.D. Research in Microelectronics and Electronics (PRIME) in 2023, and the two Best Student Paper Awards (Third Place) at the IEEE Radio Frequency Integrated Circuits (RFIC) Symposium in 2025. He was recognized as the top five Associate Editors in 2020, the five highest performing associate editors in 2021, one of the three best associate editors in 2022, one of the three best reviewers in 2022, of IEEE TVLSI Systems, one of the inaugural SSCS reviewer rewards of the IEEE Solid-State Circuits Society (SSCS) in 2023, and one of the outstanding reviewer rewards of IEEE SSCS in 2024. He has served as an Associate Editor for IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (TVLSI) SYSTEMS since 2019. He served as a Subject Editor in 2022 for Circuits and Systems of *IET Electronics Letters* (EL), an Associate Editor from 2020 to 2023 of IET EL, a Guest Editor from 2022 to 2023 of IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS in 2022, and a Guest Editor from 2021 to 2022 of IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—II: EXPRESS BRIEFS. He has served as an Editor for *International Journal of Circuit Theory and Applications* (IJCTA) since 2020. He served as the Vice-Chair from 2019 to 2021 and the Chair from 2021 to 2023 for the IEEE Macau CAS Chapter and the technical program committees of several international conferences.



Ming Li received the B.S. degree in electronic information from Nanjing Normal University, Nanjing, China, in 2023. He is currently pursuing the Ph.D. degree with the Key Laboratory of Microelectronics Device and Integrated Technology, Institute of Microelectronics of the Chinese Academy of Sciences, Beijing, China.

His current research interests include SRAM-based energy-efficient neural network accelerators, SRAM-based computing-in-memory macro design, and computing-in-memory systems.



Hao Wu received the B.S. degree in integrated circuits and integrated systems from Xidian University, Xi'an, China. He is currently pursuing the Ph.D. degree with the Key Laboratory of Microelectronics Device and Integrated Technology, Institute of Microelectronics, Chinese Academy of Sciences, Beijing, China.

His research interests include SRAM-based in-memory computing processor design and algorithm–architecture codesign.



Bingxin Zhang received the B.S. degree in electronic science and technology from Beijing University of Chemical Technology, Beijing, China, in 2023. He is currently pursuing the M.S. degree in electronic information with the Institute of Microelectronics of the Chinese Academy of Sciences, Beijing.

His current research interests include SRAM-based high-efficiency neural network accelerators, SRAM-based computing-in-memory macro design, and computing-in-memory architectures.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:57 UTC from IEEE Xplore. Restrictions apply.



Rui Zhang received the B.S. degree in electronic information from Dalian University of Technology, Dalian, China, in 2024. He is currently pursuing the M.S. degree with the Key Laboratory of Microelectronics Device and Integrated Technology, Institute of Microelectronics of the Chinese Academy of Sciences, Beijing, China.

His current research interests include SRAM-based computing-in-memory macro design, energy-efficient neural network accelerators, and computer architecture.



Yiyang Yuan received the B.S. degree in electronic and information engineering from Beijing Institute of Technology, Beijing, China, in 2021. He is currently pursuing the Ph.D. degree with the Institute of Microelectronics of the Chinese Academy of Science, Beijing.

His research interests include compute-in-memory, resistive random access memory, artificial intelligence, and brain–machine interface.



Yiming Yang received the B.S. degree in electronic science and technology from Beijing Institute of Technology, Beijing, China, in 2020. He is currently pursuing the Ph.D. degree in the direction of analog integrated circuit with Beijing Institute of Technology, Beijing.

His research interests include analog and mixed-signal computing and data processing.



Jinshan Yue (Member, IEEE) received the B.S. and Ph.D. degrees from the Department of Electronic Engineering, Tsinghua University, Beijing, China, in 2016 and 2021, respectively.

He is currently a Post-Doctoral Researcher and an Assistant Researcher with the Institute of Microelectronics, Chinese Academy of Sciences, Beijing. His current research interests include energy-efficient neural network processors, non-volatile memory, and computing-in-memory systems.

Dr. Yue served as a TPC Member for IEEE A-SSCC2023 and ICCAD2023.



Xinghua Wang received the B.Eng. and Ph.D. degrees from Beijing Institute of Technology (BIT), Beijing, China, in 2004 and 2010, respectively.

She is currently an Associate Professor with the School of Integrated Circuits and Electronics, BIT. Her research interests include analog, radio frequency (RF), and computing-in-memory.



Xiaoran Li received the B.Eng. degree in telecommunication engineering from Beijing University of Posts and Telecommunications, Beijing, China, in 2011, and the M.E. and Ph.D. degrees in electronics science and technology from Beijing Institute of Technology, Beijing, in 2013 and 2017, respectively.

She was a Visiting Scholar with the Department of Electrical and Computer Engineering, Duke University, Durham, NC, USA, from 2015 to 2016. She was a Post-Doctoral Research Fellow with Beijing Institute of Technology, where she is currently an Assistant Professor. Her research interests include analog, radio frequency, millimeter-wave integrated circuits design, computing in memory, sparse sensing, and communications.



Feng Zhang (Senior Member, IEEE) received the B.S. degree from Beijing Institute of Technology, Beijing, China, in 2000, and the M.S. and Ph.D. degrees from the Institute of Microelectronics, Chinese Academy of Sciences (IMECAS), Beijing, in 2002 and 2005, respectively.

In 2010, he joined the IMECAS as a Professor. He has published more than 80 international journal articles and conference papers. His research interests include memory, low-power digital circuits, and related hardware security techniques when applied to the "smart Internet of Things (IoT)" field.

Dr. Zhang serves as a Technical Committee Member for the IEEE Circuits and Systems Society (CASS).

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:16:57 UTC from IEEE Xplore. Restrictions apply.